In [ ]:
```
https://www.kaggle.com/datasets/adityaakuskar/accident-prediction-dataset
```

1.Project Prediction

- Our project predicts the severity of a road traffic accident as Slight Injury, Serious Injury, or Fatal Injury using the driver's information, vehicle details, road conditions, weather conditions, accident circumstances, and other related factors.

1.1 Business Problems:

- To analyze road traffic accident data and identify the factors associated with accident severity, such as driver characteristics, road conditions, weather, vehicle conditions, and accident circumstances.

2.Dataset Understanding

In [1]:
```
import pandas as pd
```

In [2]:
```
import numpy as np
```

In [3]:
```
import matplotlib.pyplot as plt
```

In [4]:
```
import seaborn as sns
```

In [5]:
```
df = pd.read_csv(r"C:\Users\mundr\Downloads\RTA Dataset.csv")
df.head(1)
```

Out[5]:

	Time	Day_of_week	Age_band_of_driver	Sex_of_driver	Educational_level	Vehi
0	17:02:00	Monday	18-30	Male	Above high school	

1 rows × 32 columns

In [6]:
```
df.shape
```

Out[6]:
```
(12316, 32)
```

In [7]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12316 entries, 0 to 12315
Data columns (total 32 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Time                         12316 non-null  str
 1   Day_of_week                  12316 non-null  str
 2   Age_band_of_driver           12316 non-null  str
 3   Sex_of_driver                12316 non-null  str
 4   Educational_level            11575 non-null  str
 5   Vehicle_driver_relation      11737 non-null  str
 6   Driving_experience           11487 non-null  str
 7   Type_of_vehicle              11366 non-null  str
 8   Owner_of_vehicle             11834 non-null  str
 9   Service_year_of_vehicle      8388 non-null   str
 10  Defect_of_vehicle            7889 non-null   str
 11  Area_accident_occured        12077 non-null  str
 12  Lanes_or_Medians             11931 non-null  str
 13  Road_allignment              12174 non-null  str
 14  Types_of_Junction            11429 non-null  str
 15  Road_surface_type            12144 non-null  str
 16  Road_surface_conditions      12316 non-null  str
 17  Light_conditions             12316 non-null  str
 18  Weather_conditions           12316 non-null  str
 19  Type_of_collision            12161 non-null  str
 20  Number_of_vehicles_involved  12316 non-null  int64
 21  Number_of_casualties         12316 non-null  int64
 22  Vehicle_movement             12008 non-null  str
 23  Casualty_class               12316 non-null  str
 24  Sex_of_casualty              12316 non-null  str
 25  Age_band_of_casualty         12316 non-null  str
 26  Casualty_severity            12316 non-null  str
 27  Work_of_casuality            9118 non-null   str
 28  Fitness_of_casuality         9681 non-null   str
 29  Pedestrian_movement          12316 non-null  str
 30  Cause_of_accident            12316 non-null  str
 31  Accident_severity            12316 non-null  str
dtypes: int64(2), str(30)
memory usage: 3.0 MB
```

In [8]:
```
df.describe()
```

Out[8]:

	Number_of_vehicles_involved	Number_of_casualties
count	12316.000000	12316.000000
mean	2.040679	1.548149
std	0.688790	1.007179
min	1.000000	1.000000
25%	2.000000	1.000000
50%	2.000000	1.000000
75%	2.000000	2.000000
max	7.000000	8.000000

In [9]:
```
df.columns
```

Out[9]:
```
Index(['Time', 'Day_of_week', 'Age_band_of_driver', 'Sex_of_driver',
       'Educational_level', 'Vehicle_driver_relation', 'Driving_experien
ce',
       'Type_of_vehicle', 'Owner_of_vehicle', 'Service_year_of_vehicle',
       'Defect_of_vehicle', 'Area_accident_occured', 'Lanes_or_Medians',
       'Road_allignment', 'Types_of_Junction', 'Road_surface_type',
       'Road_surface_conditions', 'Light_conditions', 'Weather_condition
s',
       'Type_of_collision', 'Number_of_vehicles_involved',
       'Number_of_casualties', 'Vehicle_movement', 'Casualty_class',
       'Sex_of_casualty', 'Age_band_of_casualty', 'Casualty_severity',
       'Work_of_casuality', 'Fitness_of_casuality', 'Pedestrian_movemen
t',
       'Cause_of_accident', 'Accident_severity'],
      dtype='str')
```

3.Data Cleaning

In [10]:
```
df.isnull().mean() * 100
```

Out[10]:
```
Time                            0.000000
Day_of_week                     0.000000
Age_band_of_driver              0.000000
Sex_of_driver                   0.000000
Educational_level               6.016564
Vehicle_driver_relation         4.701202
Driving_experience              6.731082
Type_of_vehicle                 7.713543
Owner_of_vehicle                3.913608
Service_year_of_vehicle        31.893472
Defect_of_vehicle              35.945112
Area_accident_occured           1.940565
Lanes_or_Medians                3.126015
Road_allignment                 1.152972
Types_of_Junction               7.202014
Road_surface_type               1.396557
Road_surface_conditions         0.000000
Light_conditions                0.000000
Weather_conditions              0.000000
Type_of_collision               1.258525
Number_of_vehicles_involved     0.000000
Number_of_casualties            0.000000
Vehicle_movement                2.500812
Casualty_class                  0.000000
Sex_of_casualty                 0.000000
Age_band_of_casualty            0.000000
Casualty_severity               0.000000
Work_of_casuality              25.966223
Fitness_of_casuality           21.394933
Pedestrian_movement             0.000000
Cause_of_accident               0.000000
Accident_severity               0.000000
dtype: float64
```

In [11]:
```
num = df.select_dtypes(include = ["int64","float64"]).columns
num
```

Out[11]:
```
Index(['Number_of_vehicles_involved', 'Number_of_casualties'], dtype='st
r')
```

In [12]:
```
import warnings
warnings.filterwarnings("ignore")
```

In [13]:
```
cat =  df.select_dtypes(include="object").columns
cat
```

Out[13]:
```
Index(['Time', 'Day_of_week', 'Age_band_of_driver', 'Sex_of_driver',
       'Educational_level', 'Vehicle_driver_relation', 'Driving_experien
ce',
       'Type_of_vehicle', 'Owner_of_vehicle', 'Service_year_of_vehicle',
       'Defect_of_vehicle', 'Area_accident_occured', 'Lanes_or_Medians',
       'Road_allignment', 'Types_of_Junction', 'Road_surface_type',
       'Road_surface_conditions', 'Light_conditions', 'Weather_condition
s',
       'Type_of_collision', 'Vehicle_movement', 'Casualty_class',
       'Sex_of_casualty', 'Age_band_of_casualty', 'Casualty_severity',
       'Work_of_casuality', 'Fitness_of_casuality', 'Pedestrian_movemen
t',
       'Cause_of_accident', 'Accident_severity'],
      dtype='str')
```

In [14]:
```
for i in num:
    df[i] = df[i].fillna(df[i].median())
```

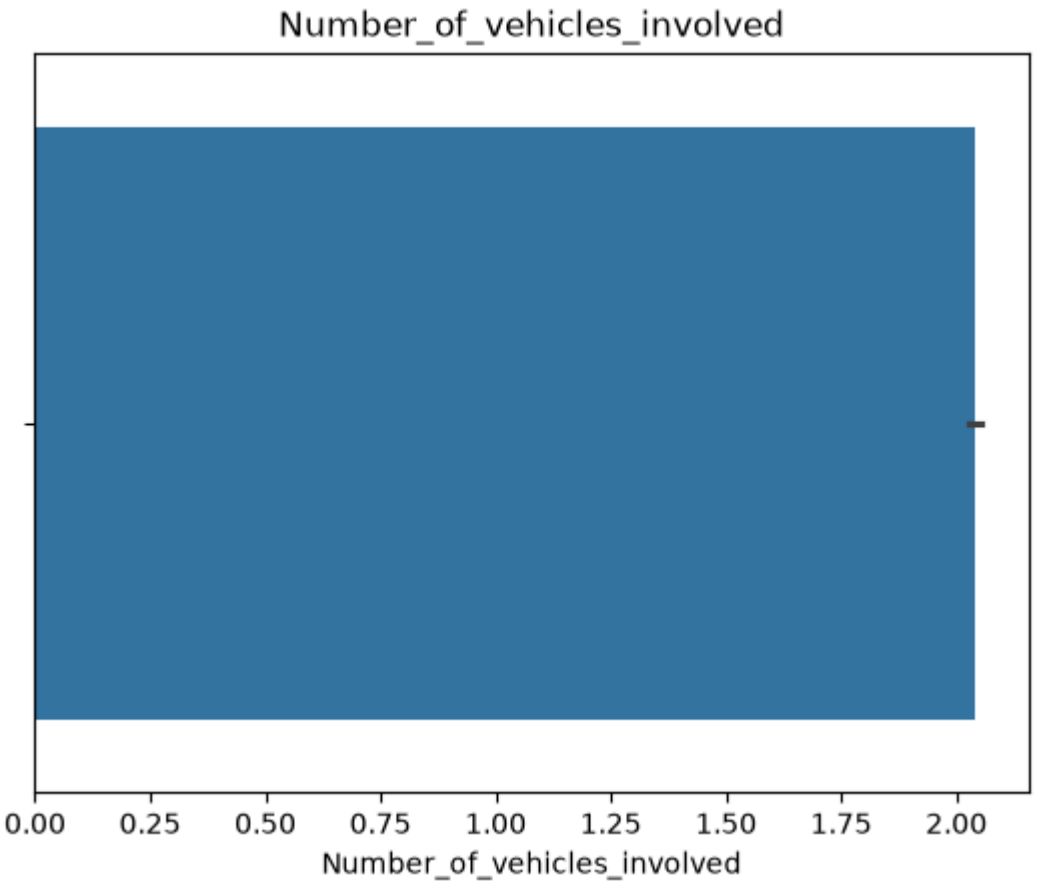
In [15]:
```
for j in cat:
    df[j] = df[j].fillna(df[j].mode()[0])
```

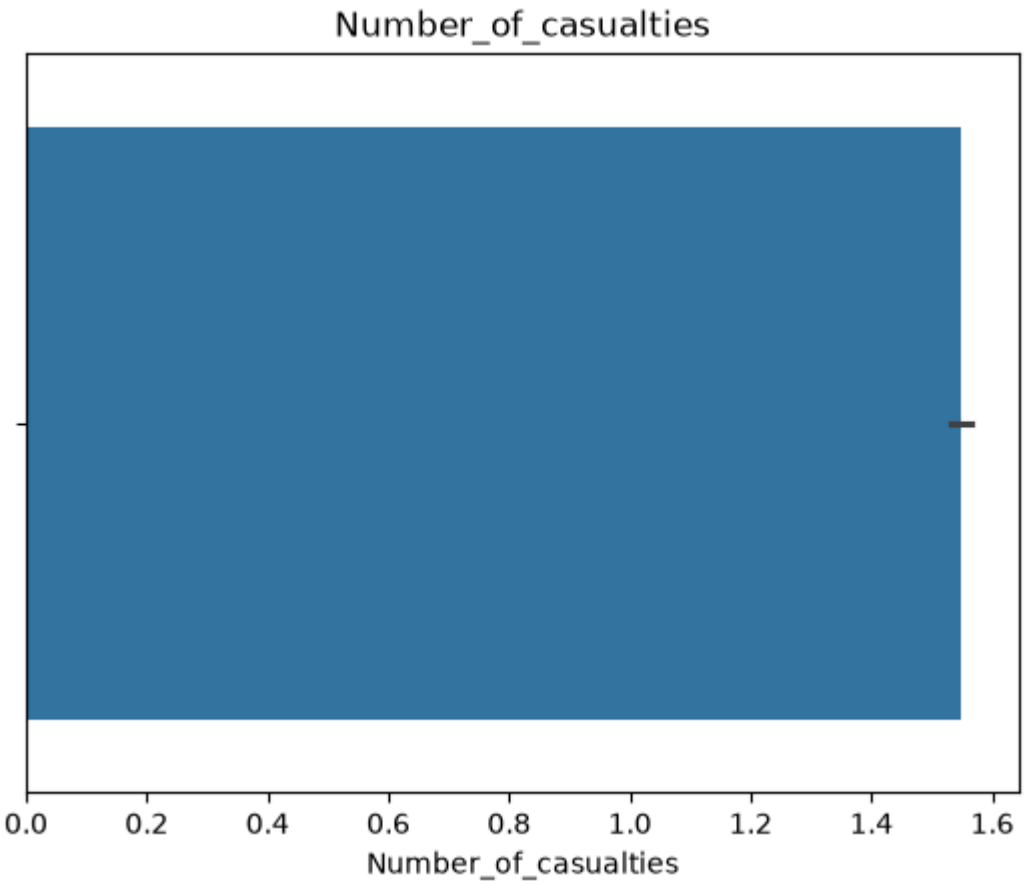
In [16]:
```
df.isnull().mean() * 100
```

Out[16]:
```
Time                           0.0
Day_of_week                    0.0
Age_band_of_driver             0.0
Sex_of_driver                  0.0
Educational_level              0.0
Vehicle_driver_relation        0.0
Driving_experience             0.0
Type_of_vehicle                0.0
Owner_of_vehicle               0.0
Service_year_of_vehicle        0.0
Defect_of_vehicle              0.0
Area_accident_occured          0.0
Lanes_or_Medians               0.0
Road_allignment                0.0
Types_of_Junction              0.0
Road_surface_type              0.0
Road_surface_conditions        0.0
Light_conditions               0.0
Weather_conditions             0.0
Type_of_collision              0.0
Number_of_vehicles_involved    0.0
Number_of_casualties           0.0
Vehicle_movement               0.0
Casualty_class                 0.0
Sex_of_casualty                0.0
Age_band_of_casualty           0.0
Casualty_severity              0.0
Work_of_casuality              0.0
Fitness_of_casuality           0.0
Pedestrian_movement            0.0
Cause_of_accident              0.0
Accident_severity              0.0
dtype: float64
```

4.Checking Outliers for numerical columns

In [17]:
```
for i in num:
    sns.barplot(data = df , x = i)
    plt.title(i)
    plt.show(i)
```





In [18]:

```
def outlier(df , columns):
    q1 = df[i].quantile(0.25)
    q3 = df[i].quantile(0.75)

    iqr = q3 -  q1
```

```
    lb = q1 - 1.5 * iqr
    ub = q3 + 1.5 * iqr

    return df[(df[i] < lb) | (df[i] > ub)]
```

In [19]:
```
for i in num:
    print("\n",i)
    print(len(outlier(df,i)))
```

```
 Number_of_vehicles_involved
3976

 Number_of_casualties
720
```

5.Treatment of outlier

In [20]:
```
from feature_engine.outliers import Winsorizer
```

In [21]:
```
win_obj = Winsorizer(capping_method= "iqr",tail="both")
win_obj
```

Out[21]:
▾ Winsorizer ⓘ

▸ Parameters

In [22]:
```
win = Winsorizer(
    capping_method="iqr",
    tail="both",
    fold=1.5,
    variables=["Number_of_casualties"])

df[["Number_of_casualties"]] = win.fit_transform(df[["Number_of_casualtie
```

In [23]:
```
for i in num:
    print("\n",i)
    print(len(outlier(df,i)))
```

```
 Number_of_vehicles_involved
3976

 Number_of_casualties
0
```

6. checking for duplicates

In [24]:
```
df.duplicated().sum()
```

Out[24]:
```
np.int64(0)
```

7.checking data inconsistent ot not

7.1 checking for Numerical Columns

In [25]:
```
for i in num:
    print("\n",i)
    print(df[i].unique())
```

```
 Number_of_vehicles_involved
[2 1 3 6 4 7]

 Number_of_casualties
[2.  1.  3.  3.5]
```

7.2 Converting Float to Int

In [26]:
```
df["Number_of_casualties"] = df["Number_of_casualties"].astype(int)
```

In [27]:
```
df[['Number_of_vehicles_involved','Number_of_casualties']].dtypes
```

Out[27]:
```
Number_of_vehicles_involved    int64
Number_of_casualties           int64
dtype: object
```

7.3 checking for Categorical Columns

In [28]:
```
for j in cat:
    print("\n",j)
    print(df[j].unique())
```

```
 Time
<StringArray>
['17:02:00', '01:06:00', '14:15:00', '17:30:00', '17:20:00', '14:40:00',
 '17:45:00', '22:45:00', '08:20:00', '15:10:00',
 ...
 '23:55:00', '15:58:00', '10:26:00', '05:53:00', '08:34:00', '22:11:00',
 '07:02:00', '07:24:00', '19:18:00', '02:47:00']
Length: 1074, dtype: str

 Day_of_week
<StringArray>
['Monday', 'Sunday', 'Friday', 'Wednesday', 'Saturday', 'Thursday', 'Tuesd
ay']
Length: 7, dtype: str

 Age_band_of_driver
<StringArray>
['18-30', '31-50', 'Under 18', 'Over 51', 'Unknown']
Length: 5, dtype: str

 Sex_of_driver
<StringArray>
['Male', 'Female', 'Unknown']
Length: 3, dtype: str

 Educational_level
<StringArray>
[ 'Above high school', 'Junior high school',  'Elementary school',
        'High school',            'Unknown',         'Illiterate',
  'Writing & reading']
Length: 7, dtype: str

 Vehicle_driver_relation
<StringArray>
['Employee', 'Unknown', 'Owner', 'Other']
Length: 4, dtype: str

 Driving_experience
<StringArray>
['1-2yr', 'Above 10yr', '5-10yr', '2-5yr', 'No Licence', 'Below 1yr',
 'unknown']
Length: 7, dtype: str

 Type_of_vehicle
<StringArray>
[          'Automobile',  'Public (> 45 seats)',      'Lorry (41?100Q)',
 'Public (13?45 seats)',       'Lorry (11?40Q)',           'Long lorry',
    'Public (12 seats)',                 'Taxi',     'Pick up upto 10Q',
         'Stationwagen',         'Ridden horse',               'Other',
               'Bajaj',                'Turbo',           'Motorcycle',
      'Special vehicle',              'Bicycle']
Length: 17, dtype: str

 Owner_of_vehicle
<StringArray>
['Owner', 'Governmental', 'Organization', 'Other']
Length: 4, dtype: str

 Service_year_of_vehicle
<StringArray>
```

```
['Above 10yr', '5-10yrs', 'Unknown', '1-2yr', '2-5yrs', 'Below 1yr']
Length: 6, dtype: str

 Defect_of_vehicle
<StringArray>
['No defect', '7', '5']
Length: 3, dtype: str

 Area_accident_occured
<StringArray>
[              'Residential areas',                   'Office areas',
            '  Recreational areas',               ' Industrial areas',
                           'Other',                  ' Church areas',
                '  Market areas',                       'Unknown',
             'Rural village areas',            ' Outside rural areas',
                ' Hospital areas',                   'School areas',
 'Rural village areasOffice areas',              'Recreational areas']
Length: 14, dtype: str

 Lanes_or_Medians
<StringArray>
['Two-way (divided with broken lines road marking)',
                                'Undivided Two way',
                                            'other',
                     'Double carriageway (median)',
                                           'One way',
  'Two-way (divided with solid lines road marking)',
                                           'Unknown']
Length: 7, dtype: str

 Road_allignment
<StringArray>
[               'Tangent road with flat terrain',
 'Tangent road with mild grade and flat terrain',
                                  'Escarpments',
             'Tangent road with rolling terrain',
                       'Gentle horizontal curve',
     'Tangent road with mountainous terrain and',
 'Steep grade downward with mountainous terrain',
                           'Sharp reverse curve',
   'Steep grade upward with mountainous terrain']
Length: 9, dtype: str

 Types_of_Junction
<StringArray>
['No junction',     'Y Shape',    'Crossing',     'O Shape',       'Othe
r',
     'Unknown',     'T Shape',     'X Shape']
Length: 8, dtype: str

 Road_surface_type
<StringArray>
[                   'Asphalt roads',                      'Earth roads',
 'Asphalt roads with some distress',                    'Gravel roads',
                            'Other']
Length: 5, dtype: str

 Road_surface_conditions
<StringArray>
['Dry', 'Wet or damp', 'Snow', 'Flood over 3cm. deep']
```

```
Length: 4, dtype: str

 Light_conditions
<StringArray>
[               'Daylight',   'Darkness - lights lit',
  'Darkness - no lighting', 'Darkness - lights unlit']
Length: 4, dtype: str

 Weather_conditions
<StringArray>
[           'Normal',           'Raining', 'Raining and Windy',
            'Cloudy',             'Other',             'Windy',
              'Snow',           'Unknown',       'Fog or mist']
Length: 9, dtype: str

 Type_of_collision
<StringArray>
['Collision with roadside-parked vehicles',
          'Vehicle with vehicle collision',
         'Collision with roadside objects',
                 'Collision with animals',
                               'Other',
                             'Rollover',
                    'Fall from vehicles',
              'Collision with pedestrians',
                            'With Train',
                               'Unknown']
Length: 10, dtype: str

 Vehicle_movement
<StringArray>
[     'Going straight',              'U-Turn',     'Moving Backward',
             'Turnover',       'Waiting to go',         'Getting off',
            'Reversing',             'Unknown',              'Parked',
             'Stopping',          'Overtaking',               'Other',
 'Entering a junction']
Length: 13, dtype: str

 Casualty_class
<StringArray>
['na', 'Driver or rider', 'Pedestrian', 'Passenger']
Length: 4, dtype: str

 Sex_of_casualty
<StringArray>
['na', 'Male', 'Female']
Length: 3, dtype: str

 Age_band_of_casualty
<StringArray>
['na', '31-50', '18-30', 'Under 18', 'Over 51', '5']
Length: 6, dtype: str

 Casualty_severity
<StringArray>
['na', '3', '2', '1']
Length: 4, dtype: str

 Work_of_casuality
<StringArray>
```

```
[       'Driver',         'Other',    'Unemployed',      'Employee',
 'Self-employed',       'Student',       'Unknown']
Length: 7, dtype: str

 Fitness_of_casuality
<StringArray>
['Normal', 'Deaf', 'Other', 'Blind', 'NormalNormal']
Length: 5, dtype: str

 Pedestrian_movement
<StringArray>
[
'Not a Pedestrian',

'Crossing from driver's nearside',
                                                          'Crossing from ne
arside - masked by parked or statioNot a Pedestrianry vehicle',

'Unknown or other',
                                                          'Crossing from of
fside - masked by  parked or statioNot a Pedestrianry vehicle',
                                                        'In carriageway, s
tatioNot a Pedestrianry - not crossing  (standing or playing)',

'Walking along in carriageway, back to traffic',

'Walking along in carriageway, facing traffic',
 'In carriageway, statioNot a Pedestrianry - not crossing  (standing or pl
aying) - masked by parked or statioNot a Pedestrianry vehicle']
Length: 9, dtype: str

 Cause_of_accident
<StringArray>
[                     'Moving Backward',
                           'Overtaking',
            'Changing lane to the left',
           'Changing lane to the right',
                          'Overloading',
                               'Other',
               'No priority to vehicle',
            'No priority to pedestrian',
                        'No distancing',
   'Getting off the vehicle improperly',
                      'Improper parking',
                             'Overspeed',
                  'Driving carelessly',
                'Driving at high speed',
                  'Driving to the left',
                              'Unknown',
                         'Overturning',
                            'Turnover',
 'Driving under the influence of drugs',
                        'Drunk driving']
Length: 20, dtype: str

 Accident_severity
<StringArray>
['Slight Injury', 'Serious Injury', 'Fatal injury']
Length: 3, dtype: str
```

7.3.1 Remove leading/trailing spaces from all text columns

In [29]:
```
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()
```

7.3.2 Replace multiple spaces with a single space

In [30]:
```
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.replace(r"\s+", " ", regex=True)
```

7.3.3 Fix capitalization inconsistencies

In [31]:
```
df["Driving_experience"] = df["Driving_experience"].replace({"unknown": "
df["Lanes_or_Medians"] = df["Lanes_or_Medians"].replace({"other": "Other"
```

7.3.4 Fix Area_accident_occured

In [32]:
```
df["Area_accident_occured"] = df["Area_accident_occured"].replace({
    "Rural village areasOffice areas": "Rural village areas",
    "  Recreational areas": "Recreational areas",
    " Industrial areas": "Industrial areas",
    " Church areas": "Church areas",
    "  Market areas": "Market areas",
    " Outside rural areas": "Outside rural areas",
    " Hospital areas": "Hospital areas"})
```

7.3.5 Fix Type_of_vehicle

In [33]:
```
df["Type_of_vehicle"] = df["Type_of_vehicle"].replace({
    "Lorry (41?100Q)": "Lorry (41-100Q)",
    "Lorry (11?40Q)": "Lorry (11-40Q)",
    "Public (13?45 seats)": "Public (13-45 seats)"})
```

7.3.6 Fix Pedestrian_movement

In [34]:
```
df["Pedestrian_movement"] = df["Pedestrian_movement"].replace({
    "statioNot a Pedestrianry vehicle": "Not a Pedestrian",
    "parked or statioNot a Pedestrianry vehicle": "Not a Pedestrian"})
```

7.3.7 Fix Fitness_of_casuality

In [35]:
```
df["Fitness_of_casuality"] = df["Fitness_of_casuality"].replace({
    "NormalNormal": "Normal"})
```

7.3.8 Check for remaining special symbols

In [36]:
```
for col in df.select_dtypes(include="object").columns:
    special = df[col].astype(str).str.contains(
        r"[^A-Za-z0-9\s&()/.,+_\-]",
        regex=True)
```

8. Data Manipulation

1. Find the average number of casualties for each accident severity?

In [37]:
```
df.groupby('Accident_severity')[['Number_of_vehicles_involved','Number_of
```

Out[37]:

Accident_severity	Number_of_vehicles_involved	Number_of_casualties
Fatal injury	1.797468	1.987342
Serious Injury	1.895009	1.425703
Slight Injury	2.068747	1.446471

2. Find the average number of vehicles involved for each accident severity?

In [38]:
```
df.groupby("Accident_severity")["Number_of_vehicles_involved"].mean()
```

Out[38]:
```
Accident_severity
Fatal injury      1.797468
Serious Injury    1.895009
Slight Injury     2.068747
Name: Number_of_vehicles_involved, dtype: float64
```

3. Find all accidents involving more than 3 vehicles and more than 2 casualties.

In [40]:
```
df.query('Number_of_vehicles_involved > 3 and Number_of_casualties > 2')
```

Out[40]:

	Time	Day_of_week	Age_band_of_driver	Sex_of_driver	Educational_level
454	11:30:00	Tuesday	18-30	Male	Junior high school
455	11:30:00	Tuesday	18-30	Male	Junior high school
456	11:30:00	Tuesday	Under 18	Male	High school
457	11:30:00	Tuesday	Under 18	Male	Junior high school
458	11:30:00	Tuesday	Under 18	Male	Junior high school
...	...	...	...	...	...
12107	16:05:00	Tuesday	Over 51	Male	Junior high school
12108	22:05:00	Friday	18-30	Male	Junior high school
12109	22:05:00	Friday	18-30	Male	Elementary school
12110	22:05:00	Friday	18-30	Male	Elementary school
12111	22:05:00	Friday	18-30	Female	Elementary school

113 rows × 32 columns

4. Find accidents involving more than 2 vehicles and more than 2 casualties?

In [41]:
```
df.query('Number_of_vehicles_involved > 2 and Number_of_casualties > 2').
```

Out[41]:

	Time	Day_of_week	Age_band_of_driver	Sex_of_driver	Educational_level	Ve
360	16:55:00	Friday	31-50	Male	Junior high school	
361	16:55:00	Friday	31-50	Male	Junior high school	
362	16:55:00	Friday	31-50	Male	Junior high school	
363	16:55:00	Friday	Unknown	Female	Junior high school	
364	16:55:00	Friday	Unknown	Female	Elementary school	
378	18:32:00	Tuesday	18-30	Male	Junior high school	
379	18:32:00	Tuesday	18-30	Male	Junior high school	
380	18:32:00	Tuesday	31-50	Male	Junior high school	
381	18:32:00	Tuesday	Over 51	Male	Junior high school	
386	15:08:00	Monday	18-30	Male	Junior high school	

10 rows × 32 columns

5. Create a frequency table showing accident severity against road surface condition?

In [42]:
```
pd.crosstab(df['Accident_severity'],df['Road_surface_conditions'])
```

Out[42]:

Road_surface_conditions	Dry	Flood over 3cm. deep	Snow	Wet or damp
Accident_severity				
Fatal injury	121	0	0	37
Serious Injury	1332	0	4	407
Slight Injury	7887	2	66	2460

6. From the crosstab, select the 1st and 3rd accident-severity rows and 2nd and 4th road-surface-condition columns?

In [43]:
```
pd.crosstab(df['Accident_severity'],df['Road_surface_conditions']).iloc[[
```

Out[43]:

Accident_severity	Road_surface_conditions	Flood over 3cm. deep	Wet or damp
Fatal injury		0	37
Slight Injury		2	2460

7.Find the accident severity at index 10.

In [44]:
```
df.at[10, 'Accident_severity']
```

Out[44]:
```
'Serious Injury'
```

8.Find the minimum number of vehicles involved for each accident severity?

In [45]:
```
df.groupby('Accident_severity')['Number_of_vehicles_involved'].min()
```

Out[45]:
```
Accident_severity
Fatal injury      1
Serious Injury    1
Slight Injury     1
Name: Number_of_vehicles_involved, dtype: int64
```

9.Find the first 10 accidents involving 4 or more vehicles?

In [46]:
```
df.query('Number_of_vehicles_involved >= 4').iloc[0:10]
```

Out[46]:

	Time	Day_of_week	Age_band_of_driver	Sex_of_driver	Educational_level	Ve
225	09:25:00	Tuesday	Over 51	Male	Above high school	
226	09:25:00	Tuesday	Unknown	Female	Elementary school	
227	09:25:00	Tuesday	Unknown	Female	Elementary school	
228	09:25:00	Tuesday	Unknown	Female	Junior high school	
229	09:25:00	Tuesday	Unknown	Female	Junior high school	
230	09:25:00	Tuesday	Unknown	Female	Elementary school	
294	17:21:00	Thursday	18-30	Male	Junior high school	
295	17:21:00	Thursday	31-50	Male	Junior high school	
296	17:21:00	Thursday	31-50	Male	Above high school	
297	17:21:00	Thursday	31-50	Male	Junior high school	

10 rows × 32 columns

10.Find the average number of casualties for each accident severity.?

In [47]:
```
pd.pivot_table(df,index='Accident_severity',values='Number_of_casualties'
```

Out[47]:

Accident_severity	Number_of_casualties
Fatal injury	1.987342
Serious Injury	1.425703
Slight Injury	1.446471

9.Univariate Analysis :

- Analysis can be done only for one column at a time.

9.1 Numerical_univariate

For numerical columns use below graphs:

- Histogram

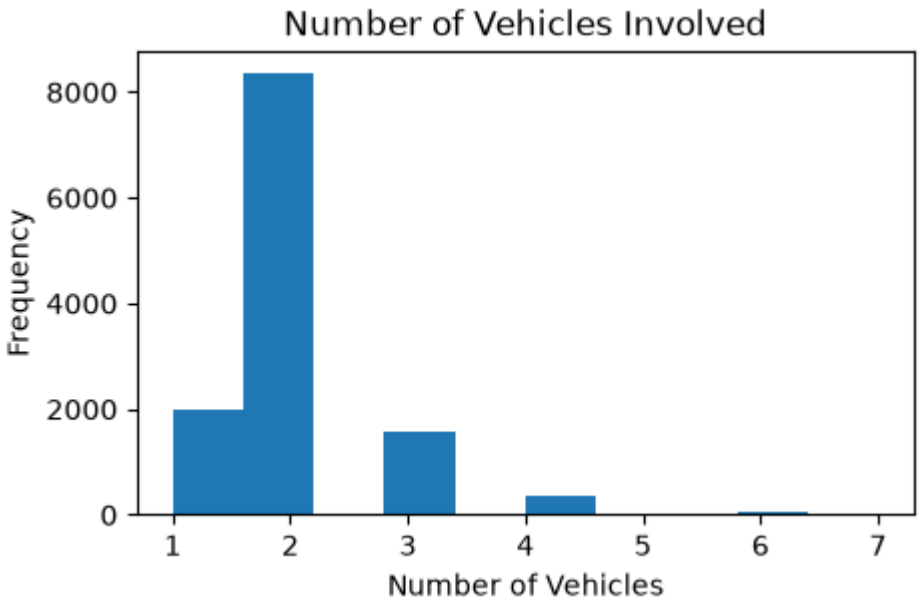
- KDE Plot
- Boxplot

- Histogram

1.Number of Vehicles Involved

In [51]:
```
plt.figure(figsize=(5,3))

df['Number_of_vehicles_involved'].plot(kind='hist')
plt.title("Number of Vehicles Involved")
plt.xlabel("Number of Vehicles")
plt.ylabel("Frequency")
plt.show()
```



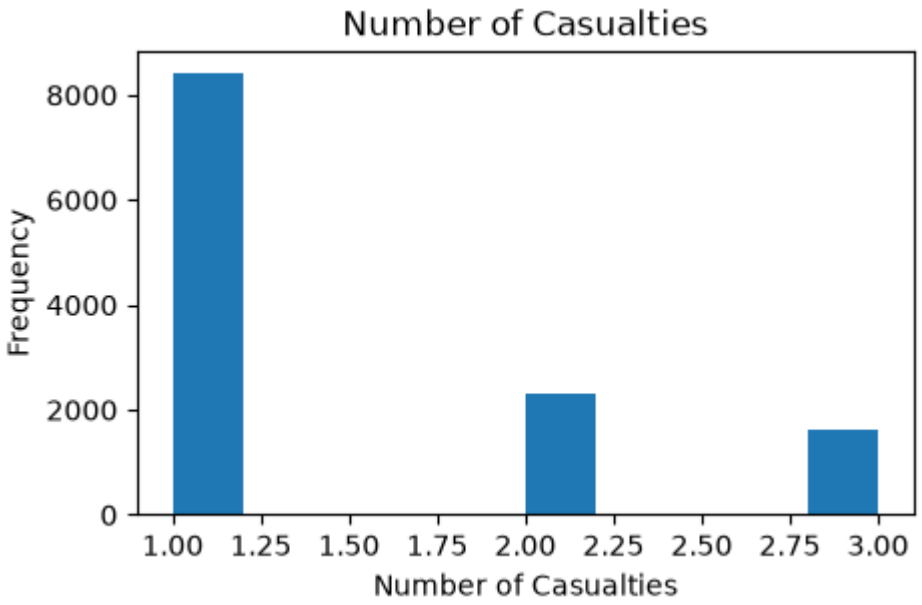
INSIGHTS

- Most accidents involve a small number of vehicles.
- Accidents involving a larger number of vehicles occur less frequently.
- This shows that multi-vehicle accidents are less common than accidents involving fewer vehicles.

2.Number of Casualties

In [52]:
```
plt.figure(figsize=(5,3))

df['Number_of_casualties'].plot(kind='hist')
plt.title("Number of Casualties")
plt.xlabel("Number of Casualties")
plt.ylabel("Frequency")
plt.show()
```



INSIGHTS

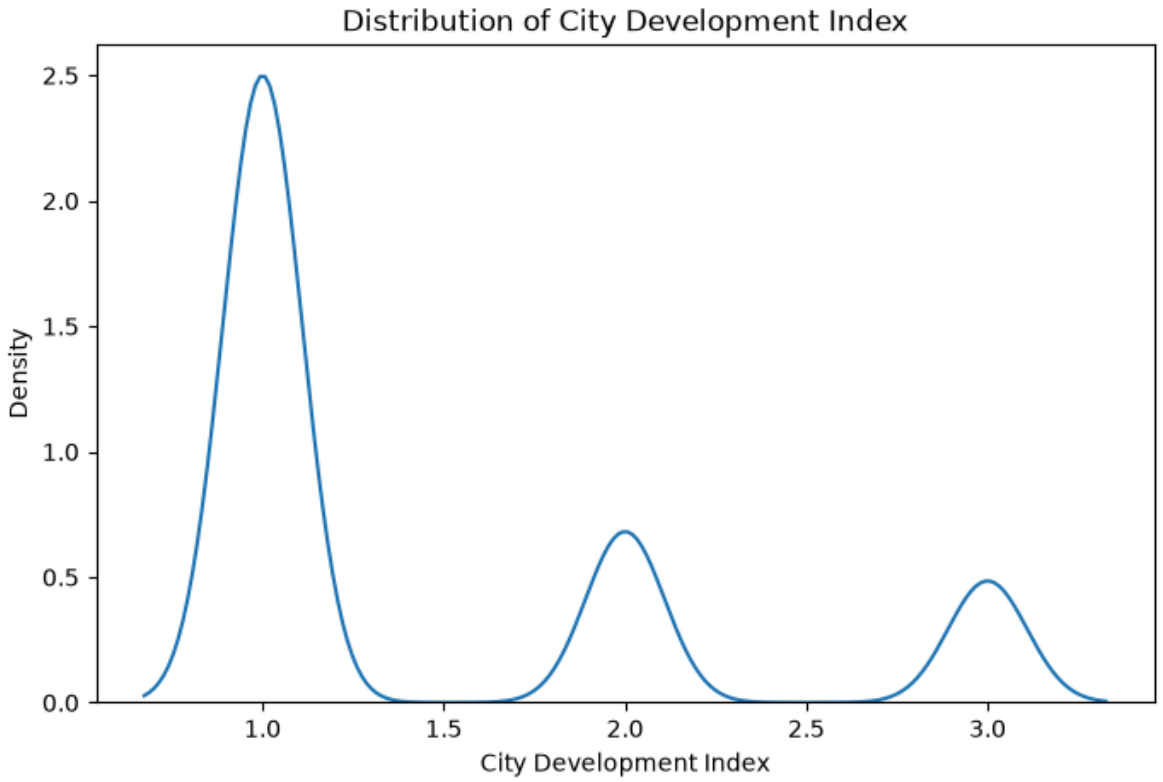
- Most accidents have a small number of casualties.
- Accidents with a higher number of casualties occur less frequently.
- The distribution helps understand the severity of casualty occurrence.

- Kde plot

3.Distribution of City Development Index

In [55]:
```
plt.figure(figsize=(8, 5))

sns.kdeplot(data=df, x='Number_of_casualties')
plt.title("Distribution of City Development Index")
plt.xlabel("City Development Index")
plt.ylabel("Density")
plt.show()
```



9.2 Categorical columns

For Categorical columns use below graphs:

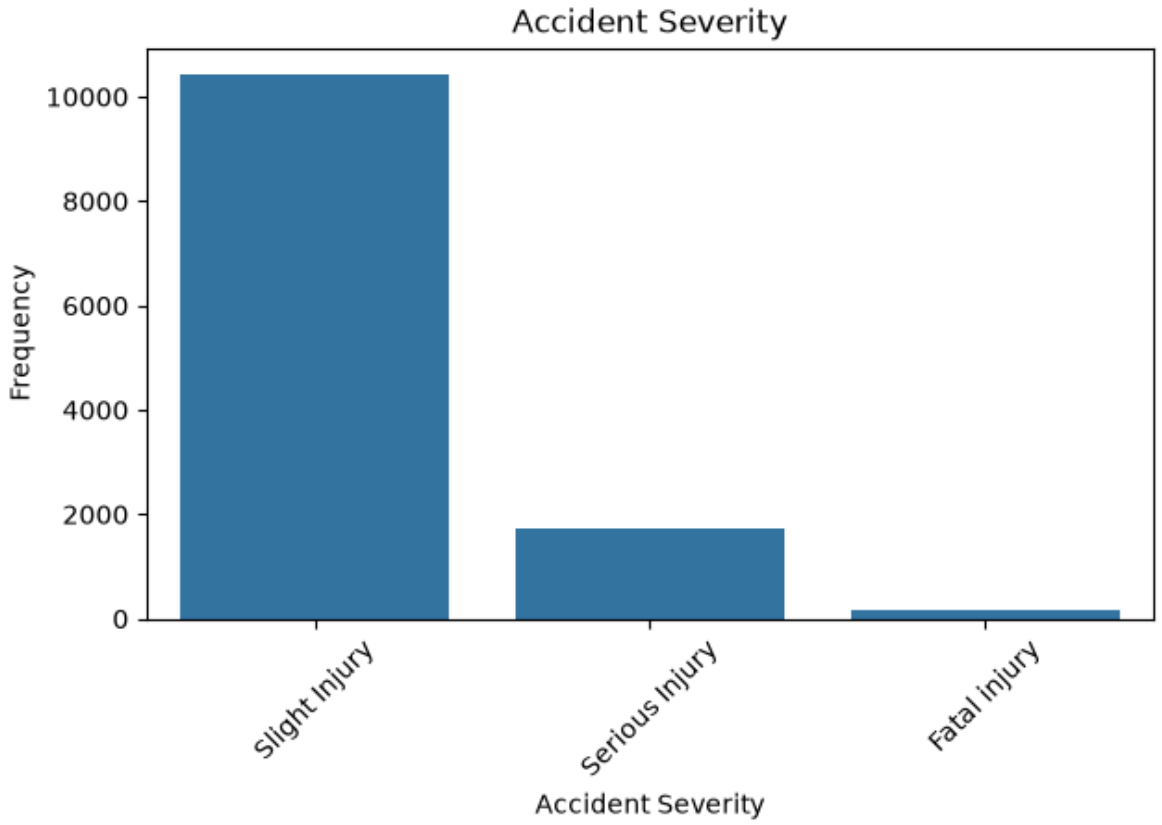
- Countplot
- Barplot
- Pie chart

- Count plot

1. Accident Severity

In [37]:
```
plt.figure(figsize=(7,4))

sns.countplot(data=df, x='Accident_severity')
plt.title("Accident Severity")
plt.xlabel("Accident Severity")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.show()
```



INSIGHTS

- The count plot shows the frequency of different accident severity levels.
- Some severity categories occur more frequently than others.
- This helps identify the most common accident severity in the dataset.

2.Driving Experience

In [ ]:
```
plt.figure(figsize=(8,4))

sns.countplot(data=df, x='Driving_experience')
plt.title("Driving Experience")
plt.xlabel("Driving Experience")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.show()
```

INSIGHTS

- The graph shows the distribution of accidents across different driving experience groups.
- Some experience groups have a higher number of accident records.
- This helps understand how accident records are distributed according to driver experience.

In [ ]:
```
Road Surface Conditions
```

In [ ]:
```
plt.figure(figsize=(7,4))

sns.countplot(data=df, x='Road_surface_conditions')

plt.title("Road Surface Conditions")
plt.xlabel("Road Surface Conditions")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.show()
```

INSIGHTS

- The graph shows accidents under different road surface conditions.
- Some road conditions occur more frequently in the dataset.
- This helps identify the common road conditions associated with accidents.

3.Weather Conditions

In [ ]:
```
plt.figure(figsize=(8,4))

sns.countplot(data=df, x='Weather_conditions')

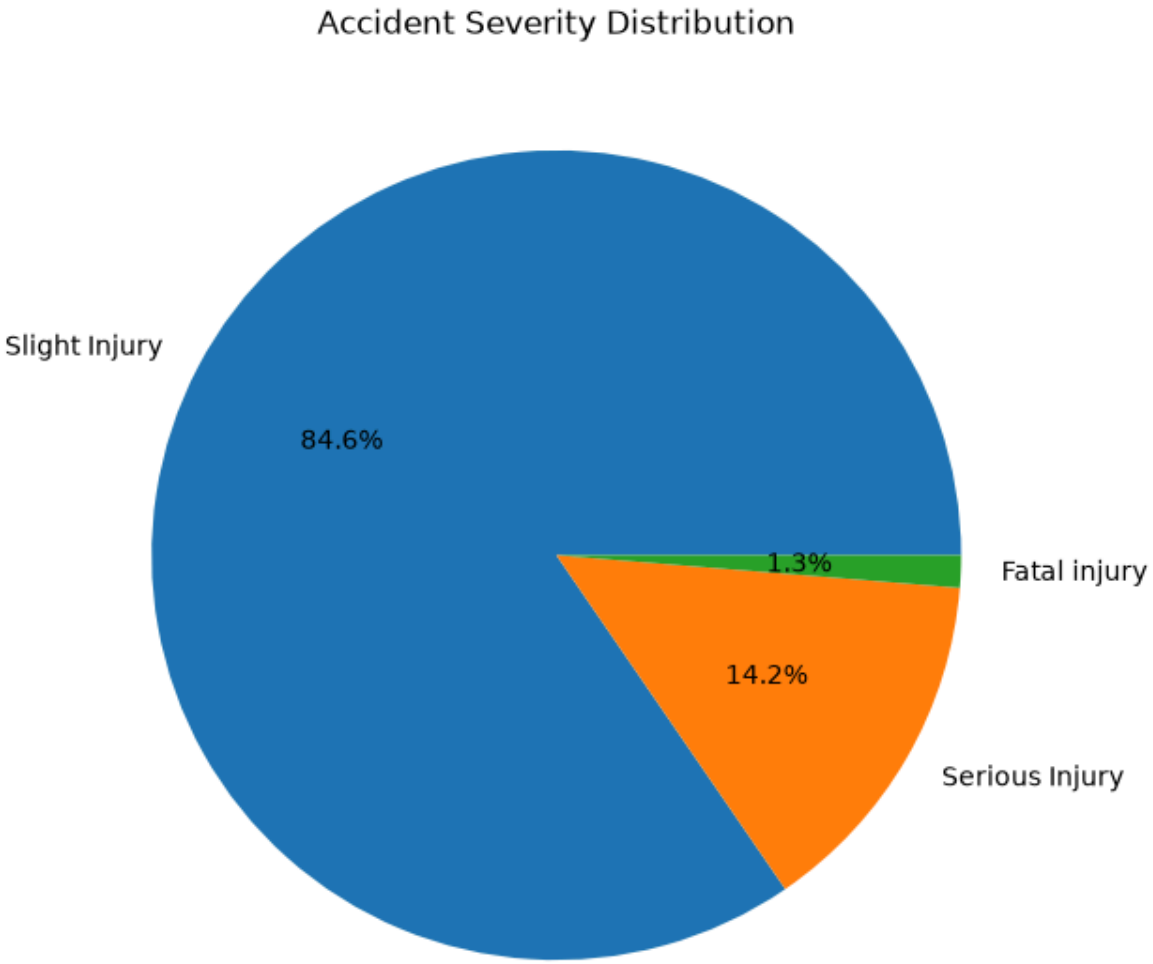
plt.title("Weather Conditions")
plt.xlabel("Weather Conditions")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.show()
```

INSIGHTS

- The graph shows the distribution of accidents under different weather conditions.
- Some weather conditions have considerably more accident records than others.
- This helps understand the weather conditions commonly present during accidents.

In [58]:
```
plt.figure(figsize=(7, 7))

df['Accident_severity'].value_counts().plot(kind='pie',autopct='%1.1f%%')
plt.title("Accident Severity Distribution")
plt.ylabel("")
plt.show()
```



INSIGHTS

- The pie chart shows the percentage distribution of accident severity.
- The largest slice represents the most common accident severity.
- It provides an easy view of the proportion of each severity category.

10. Bivariate Analysis

- Analysis can be done for two columns at a time.

10.1 Numerical × Numerical

For numerical columns use below graphs:

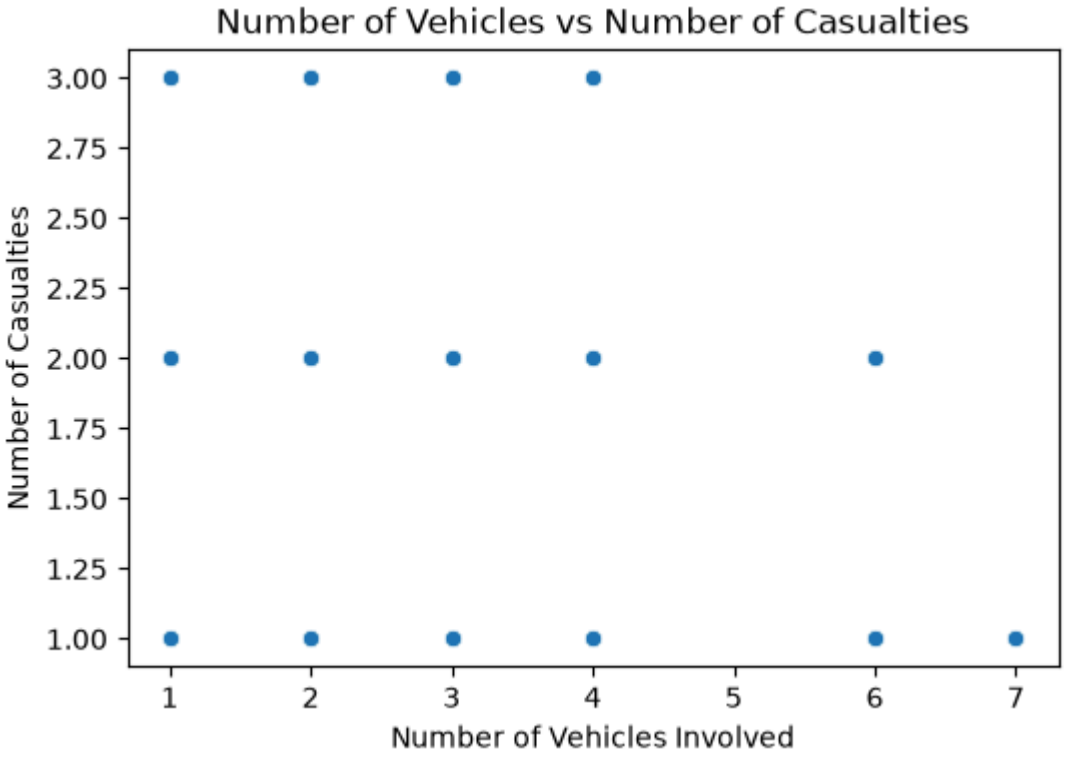
- Scatterplot

1.Number of Vehicles vs Number of Casualties

In [59]:
```
plt.figure(figsize=(6,4))

sns.scatterplot(data=df,x='Number_of_vehicles_involved',y='Number_of_casu
```

```
plt.xlabel("Number of Vehicles Involved")
plt.ylabel("Number of Casualties")
plt.title("Number of Vehicles vs Number of Casualties")
plt.show()
```



INSIGHTS

- The scatter plot shows the relationship between vehicles involved and casualties.
- The points are distributed across different casualty levels.
- It helps determine whether accidents involving more vehicles are associated with more casualties.

10.2 Numerical × Categorical

For numerical and Categorical columns use below graphs:

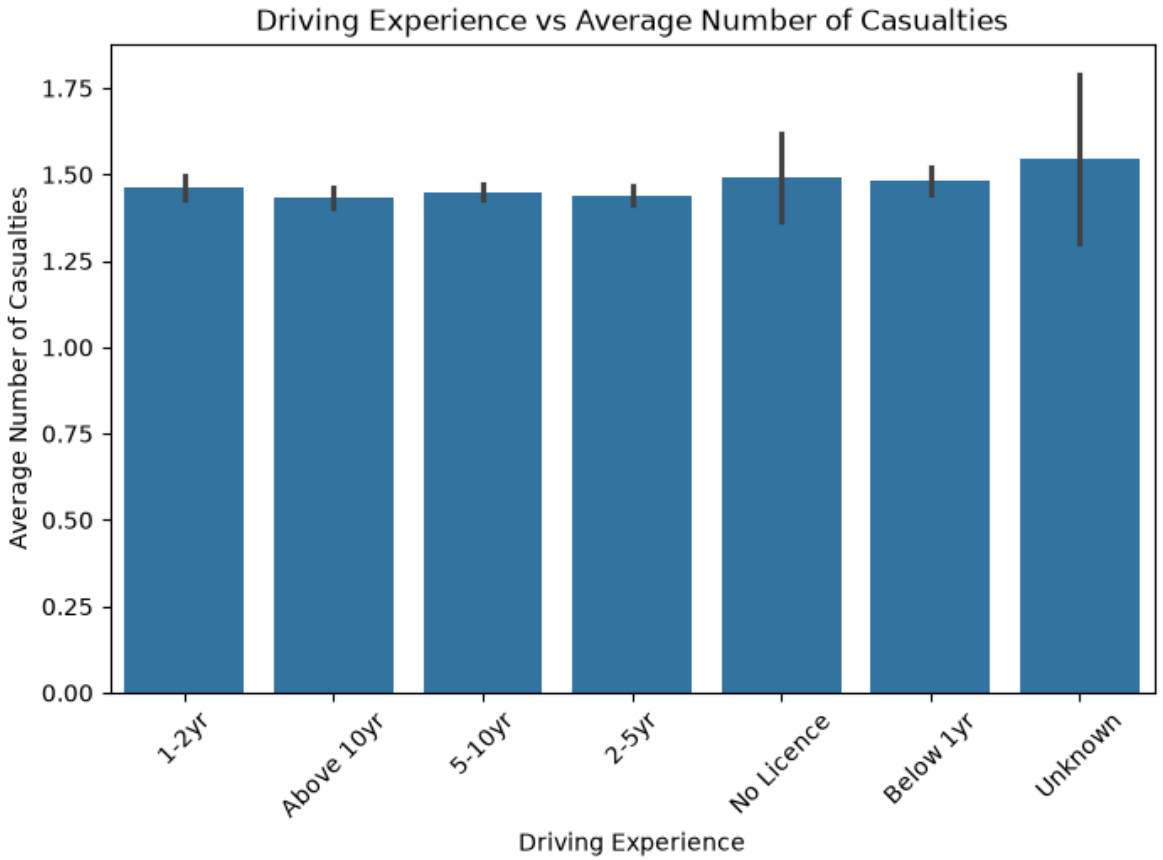
- Boxplot
- Barplot
- Line Chart
- Strip Plot
- Swarm Plot
- Violin Plot

1.Bar plot

Driving Experience × Average Casualties

In [62]:
```
plt.figure(figsize=(8, 5))

sns.barplot(data=df,x="Driving_experience",y="Number_of_casualties")
plt.title("Driving Experience vs Average Number of Casualties")
plt.xlabel("Driving Experience")
plt.ylabel("Average Number of Casualties")
plt.xticks(rotation=45)
plt.show()
```



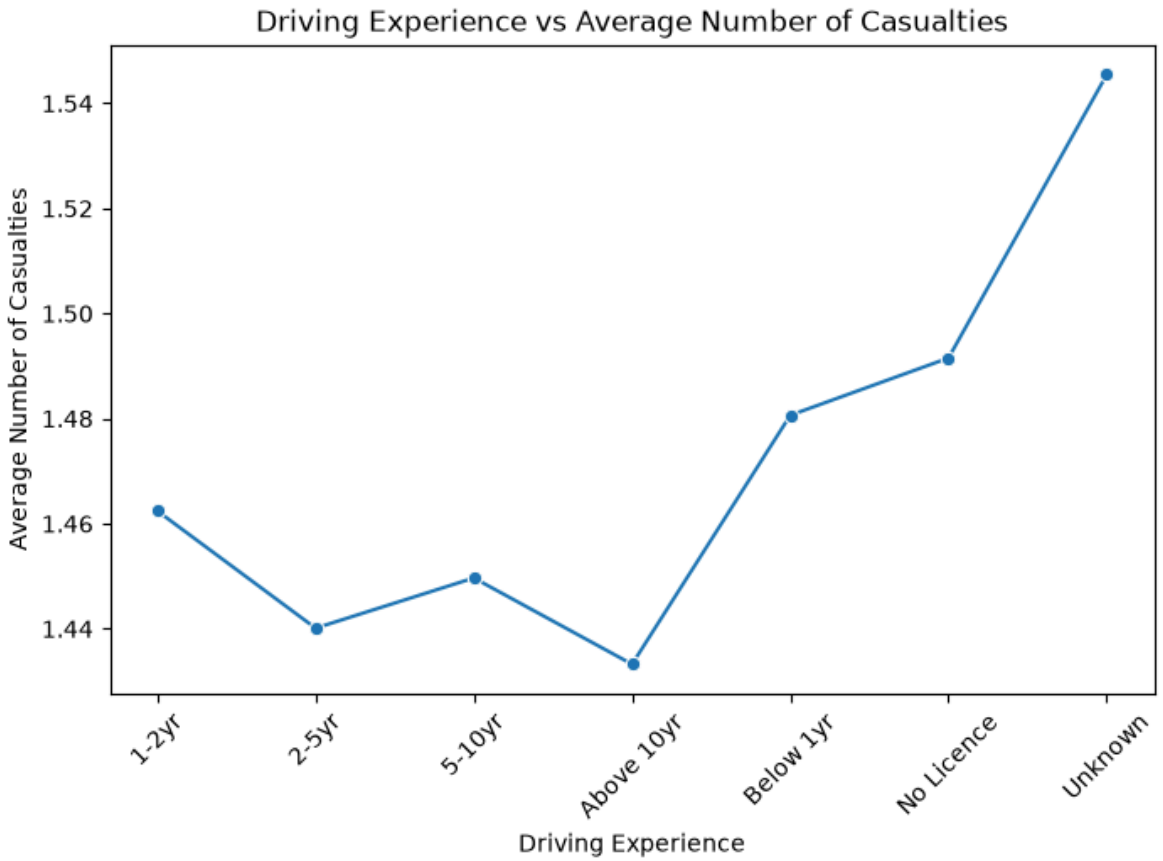
Insight

- The barplot compares the average number of casualties across driving-experience groups.
- The average casualty count varies between experience categories.
- It helps compare casualty levels for different driving-experience groups.

2.Line Chart

In [63]:
```
experience_casualties = df.groupby("Driving_experience")["Number_of_casua
plt.figure(figsize=(8, 5))
sns.lineplot(
    data=experience_casualties,
    x="Driving_experience",
    y="Number_of_casualties",
    marker="o")
plt.title("Driving Experience vs Average Number of Casualties")
plt.xlabel("Driving Experience")
plt.ylabel("Average Number of Casualties")
```

```
plt.xticks(rotation=45)
plt.show()
```



Insight

- The line chart shows how the average number of casualties changes across driving-experience groups.
- The values fluctuate between different experience categories.
- It helps observe the pattern of average casualties across the categories.

3. Strip Plot

Accident Severity vs Number of Vehicles

In [39]:
```
cat
```

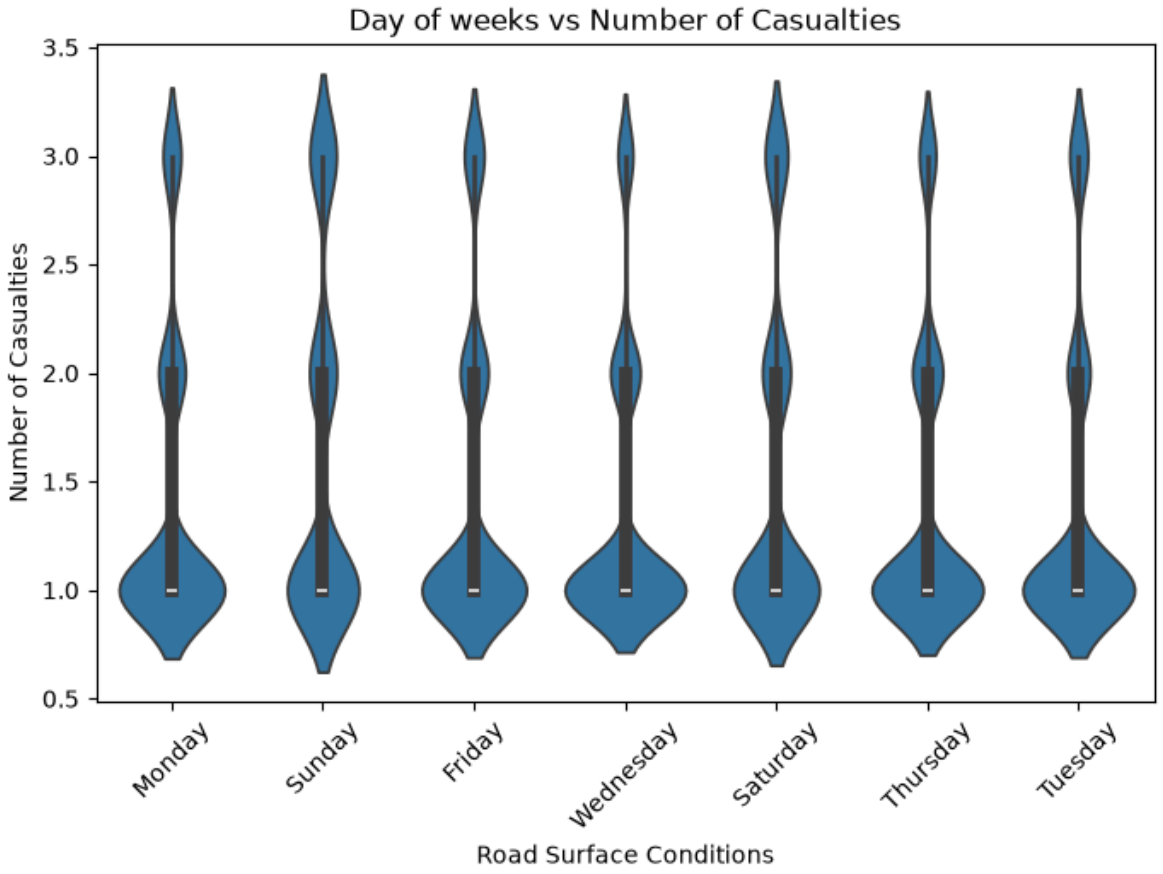
Out[39]:
```
Index(['Time', 'Day_of_week', 'Age_band_of_driver', 'Sex_of_driver',
       'Educational_level', 'Vehicle_driver_relation', 'Driving_experien
ce',
       'Type_of_vehicle', 'Owner_of_vehicle', 'Service_year_of_vehicle',
       'Defect_of_vehicle', 'Area_accident_occured', 'Lanes_or_Medians',
       'Road_allignment', 'Types_of_Junction', 'Road_surface_type',
       'Road_surface_conditions', 'Light_conditions', 'Weather_condition
s',
       'Type_of_collision', 'Vehicle_movement', 'Casualty_class',
       'Sex_of_casualty', 'Age_band_of_casualty', 'Casualty_severity',
       'Work_of_casuality', 'Fitness_of_casuality', 'Pedestrian_movemen
t',
       'Cause_of_accident', 'Accident_severity'],
      dtype='str')
```

3.Violin plot

1.Day of weeks vs Number of Casualties

In [45]:
```
plt.figure(figsize=(8, 5))

sns.violinplot(data=df,x="Day_of_week",y="Number_of_casualties")
plt.title("Day of weeks vs Number of Casualties")
plt.xlabel("Road Surface Conditions")
plt.ylabel("Number of Casualties")
plt.xticks(rotation=45)
plt.show()
```



Insights:

- The violin plot shows the distribution of Number of Casualties across different days of the week.
- Some days have a higher concentration of casualties, while others show a wider distribution.
- The plot helps identify differences and variation in casualty counts across the days of the week.

10.3 Categorical × Categorical

for Categorical and Categorical use below graphs:

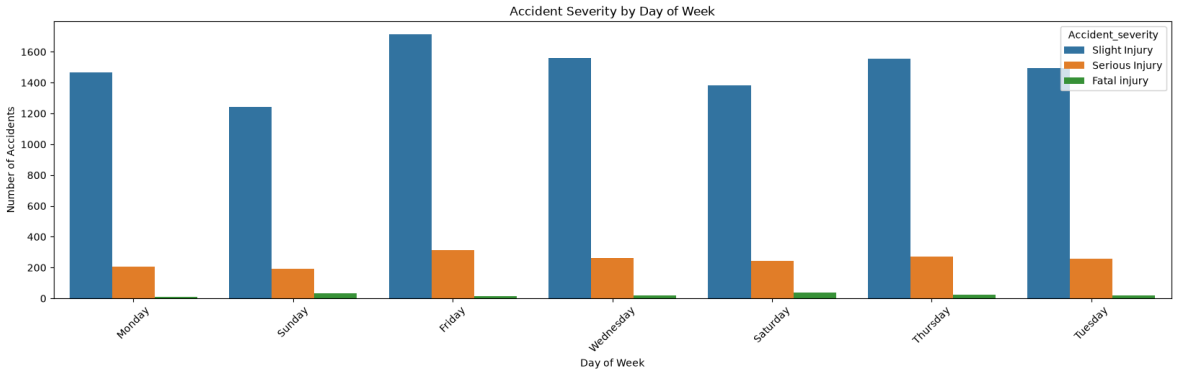
- Crosstab
- Barplot

In [46]:
```
pd.crosstab(df["Day_of_week"],df["Accident_severity"])
```

Out[46]:

Day_of_week	Fatal injury	Serious Injury	Slight Injury
Friday	16	313	1712
Monday	12	204	1465
Saturday	37	245	1384
Sunday	35	190	1242
Thursday	22	272	1557
Tuesday	17	257	1496
Wednesday	19	262	1559

In [47]:
```
plt.figure(figsize=(20, 5))
sns.countplot(data=df, x="Day_of_week",hue="Accident_severity")
plt.title("Accident Severity by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Number of Accidents")
plt.xticks(rotation=45)
plt.show()
```



Insights:

- The countplot shows the distribution of Accident Severity across different days of the week.
- Slight injury accidents are the most frequent on most days compared to serious and fatal accidents.
- The number of accidents varies by day, but Slight accidents consistently have the highest frequency.

11. Multivariate Analysis

- Analysis can be done by more than 2 columns Use below graphs for Multivariate:
- Heat map
- Pair plot
- Joint plot

11.1 Heat map

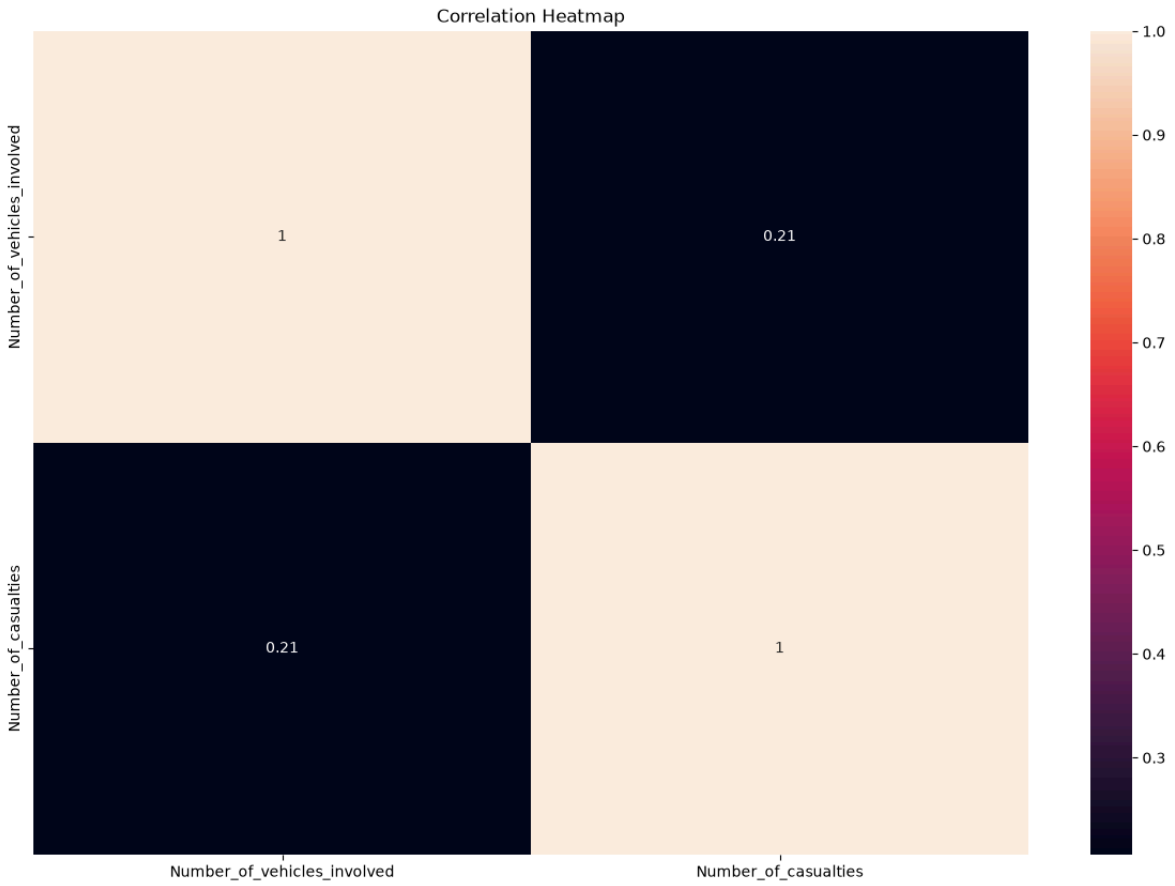
In [48]:
```
df.corr(numeric_only=True)
```

Out[48]:

	Number_of_vehicles_involved	Number_of_casualties
Number_of_vehicles_involved	1.000000	0.206632
Number_of_casualties	0.206632	1.000000

In [49]:
```
plt.figure(figsize = (15,10))
sns.heatmap(data = df.corr(numeric_only=True),annot= True)
plt.title("Correlation Heatmap")
```

Out[49]:
```
Text(0.5, 1.0, 'Correlation Heatmap')
```



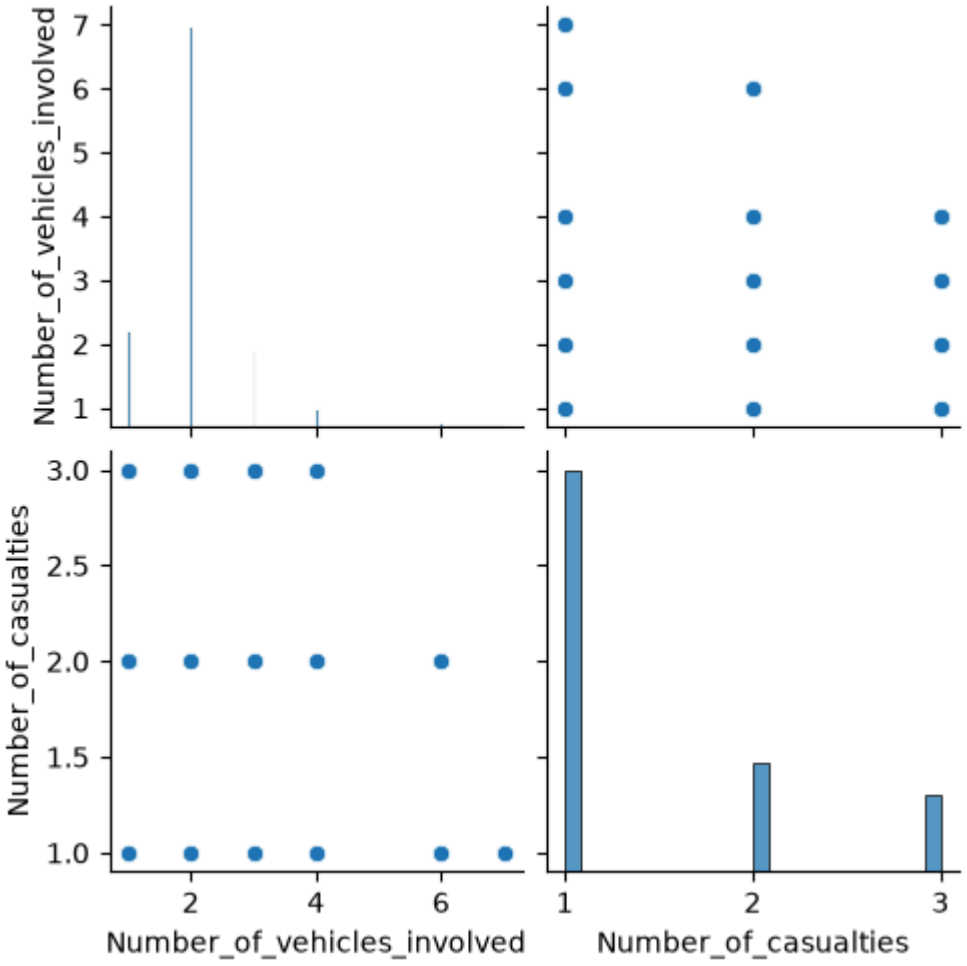
Insights:

- The heatmap shows the correlation between numerical variables in the dataset.
- Number_of_vehicles_involved and Number_of_casualties show the relationship between the number of vehicles and casualties in an accident.
- Overall, the heatmap helps identify positive, negative, and weak relationships between numerical variables.

11.2 Pairplot

In [163…

```
sns.pairplot(df[num])
plt.show()
```



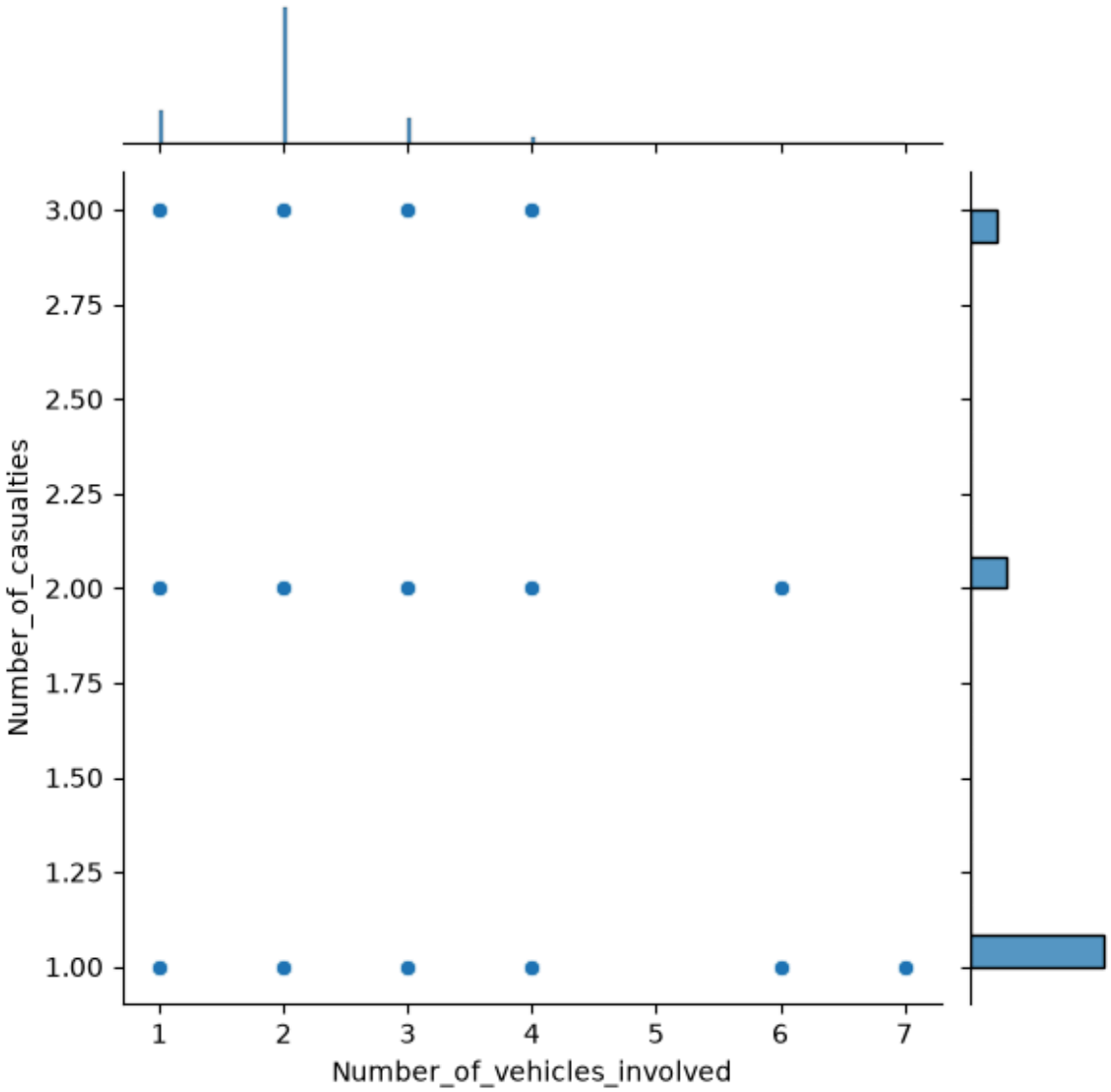
Insights:

- The pair plot shows the relationships and distributions between all numerical variables.
- Number_of_vehicles_involved and Number_of_casualties show the relationship between the numerical variables.
- The diagonal plots show the distribution of each numerical variable, while the scatter plots show their relationships.

11.3 Joint plot

In [167…

```
sns.jointplot(data=df,x="Number_of_vehicles_involved",y="Number_of_casual
plt.show()
```



Insights:

- The joint plot shows the relationship between Number_of_vehicles_involved and Number_of_casualties.
- It helps identify whether the number of casualties changes as the number of vehicles involved increases.
- The plots on the sides show the individual distribution of vehicles involved and casualties.

CONCLUSION:

What the data represents:

- This dataset represents road traffic accident information.
- It contains information about accidents such as day, time, weather conditions, road conditions, vehicles involved, casualties, drivers, pedestrians, and accident severity.
- The data helps us understand road accident patterns, accident severity, vehicle involvement, and factors associated with casualties.

What we have done:

- Checked the data types and structure.
- Checked and handled missing values.
- Checked and removed duplicate values.
- Checked and corrected data inconsistencies.
- Checked and treated outliers in numerical columns.
- Performed data manipulation using Pandas.
- Used `loc`, `iloc`, `at`, and `iat` for data selection.
- Used Boolean masking, `query()`, `groupby()`, and `value_counts()`.
- Used `pivot_table()` and `crosstab()` for data summarization.
- Created new columns and performed data transformation.
- Performed univariate analysis.
- Performed bivariate analysis.
- Performed multivariate analysis.
- Used different charts to understand the accident data.
- Created business questions and analyzed the data.
- Identified useful patterns and relationships between accident-related variables.

Improvements / Applications:

- Identify factors associated with road accidents and casualties.
- Understand accident patterns based on day, time, weather, and road conditions.
- Identify accident severity patterns.
- Understand the effect of vehicle involvement on casualties.
- Analyze accident patterns across different vehicle types and driver characteristics.
- Identify road and environmental conditions associated with more severe accidents.
- Help organizations and authorities improve road safety measures.
- Support better traffic management and accident prevention strategies.
- Use accident data to support data-driven road safety decisions.

Final Conclusion:

- Overall, the analysis helped us understand road traffic accidents, accident severity, casualties, vehicle involvement, driver characteristics, weather conditions, and road conditions.
- The analysis identified different patterns and relationships among accident-related factors.
- The insights from this dataset can help in understanding accident-prone conditions, improving road safety, supporting traffic management, and developing better accident prevention strategies.

In [ ]: